

GUI Testing as a Kimiya Program: Deterministic Scripts, Calibrated Perception, and a Path to Verified Interaction with GUI, Network, and Cloud Software

MAHMUDUL FAISAL AL AMEEN, (draft manuscript), Japan

We report a concrete instance of a general thesis: an end-to-end test of interactive software—graphical, networked, cloud-backed—is naturally expressed as a *Kimiya program*, a computation whose control flow is deterministic and whose meaning-laden steps are calibrated language-model instruments. We built a testing harness in which mouse, keyboard, vision, closed-loop audio, network fault injection, and production-cloud verification operate together over a desktop application and two concurrent browsers, and applied it to a deployed product, finding ten production bugs. We then observe that the harness had, unknowingly, implemented the Kimiya discipline: its element-locator is a select instrument, its screen-assertions are judge guards under implicit purposes, its on-screen value reads are grounded gen, and its database, cloud-API, image-hash, and audio-waveform oracles are exactly the trusted check kernel that keeps judge error from becoming a false certificate. We make the correspondence precise, transcribe a representative scenario into Kimiya surface syntax, and—using an operating point *measured* from the harness’s own run artifacts ($\beta \geq 0.976$, $\alpha \leq 0.16$, per-call locate recall ≥ 0.975)—derive a numeric reliability certificate for a passing test. The correspondence is not perfect: GUI testing stresses exactly the extension Kimiya defers, computation that *mutates an external world under adversarial timing*, and we propose the constructs a world-effecting Kimiya needs (act/observe/settle, world-frame retry, timed cost) together with the proof obligations they carry. Finally, we recast the artifact as an executable guideline: a playbook by which an autonomous language-model agent can test arbitrary GUI software under the same discipline. The larger aim is a route to *formal verification of interactive systems* that neither pretends the world is deterministic nor lets the model’s judgment go uncalibrated—using exactly as much determinism, and exactly as much language model, as each step requires, with the split made auditable by a Kimiya certificate.

Additional Key Words and Phrases: GUI testing, LLM-as-judge, program logic, calibrated oracles, end-to-end testing, verified interaction, Kimiya

1 Introduction

End-to-end testing of a graphical application is the only test layer that exercises the product as a user does, and the one teams most often abandon. Selector-bound scripts decay with every interface change; cross-platform products need a separate harness per surface; and whole classes of behavior—audio, multi-user synchronization, degraded networks, the live production backend—are declared out of scope because no convenient handle exists. Two recent capabilities reopen the problem. Vision-language models locate interface elements from natural-language descriptions reliably enough for production use, and “computer-use” agents can operate arbitrary interfaces through screenshots and synthetic input. The dominant response is to build *autonomous testing agents* that explore an application and improvise actions. Autonomy, however, is the wrong property for a regression test: an improvising agent does not run the same test twice, cannot be reviewed as an artifact, and its failures resist attribution.

This paper argues, and demonstrates on a real system, a different position. An interactive test wants to be *deterministic in its control flow and calibrated in its perception*. The steps that decide *what to do*—open this view, present these pages, then end the talk—must be a fixed, reviewable script. The steps that require *meaning*—find the button described as “the red End button,” judge whether the screen shows a live-presentation state, read the join code off a dialog—are irreducibly semantic and belong to a language model. And crucially, the *verdict* must never rest on perception

alone: a screen that merely looks right must not be able to pass a test. This is precisely the shape of a *Kimiya* program [Anonymous 2026]: deterministic structure around a triad of datasheet-carrying instruments—gen (produce), judge (discriminate), select (find)—with a trusted kernel check for what is decidable, and a program logic that transfers reliability across judged guards at *measured* error rates.

We did not set out to instantiate *Kimiya*. We built a testing harness for SeenSlide—a deployed slide-sharing product with a PyQt5 desktop capture application, a FastAPI/PostgreSQL cloud backend, and a Svelte web viewer with real-time collaboration—because the product needed testing that its unit suite could not provide. The harness drives, with one step vocabulary: a native desktop application including a GTK file dialog; two *different* real browsers concurrently under distinct user accounts; a synthetic fullscreen presenter standing in for a human; an operating-system-level virtual microphone whose audio is verified, after crossing a WebRTC mesh, at the speaker output; a fault-injecting network relay; and a live production cloud. On eighteen scenarios it found ten production bugs, three of them invisible on screen and caught only by non-visual oracles, including a database-pool defect that returned HTTP 500 to *every authenticated user* and a three-bug cascade in which each fix exposed the next.

Only afterward did the correspondence become clear: the harness had implemented the *Kimiya* discipline without naming it. That correspondence is this paper’s subject.

Contributions.

- **The correspondence (§3–§4).** A precise mapping from an interactive testing harness to *Kimiya*: locator = select, screen-assertion = judge under an implicit purpose, value-read = grounded gen, and the database/cloud/image-hash/audio oracles = the trusted check kernel. We transcribe a representative two-user scenario into *Kimiya* surface syntax.
- **A measured datasheet (§5).** We calibrate the vision locator from the harness’s own run artifacts—58 runs, 604 model calls—obtaining a conservative operating point, and use it to derive a *numeric* reliability certificate for a passing test, in the style of the *Kimiya* logic. This turns a green run from a boolean into a priced guarantee, and separates the claims a test certifies at kernel-certainty from those it certifies only at the judge’s error rate.
- **An extension for effectful, timed interaction (§6).** GUI testing stresses exactly what *Kimiya*’s core defers: computation over an external, mutating, adversarially timed world. We propose the constructs a world-effecting *Kimiya* needs—act/observe/settle, a world-frame retry rule, and timed cost—and state the proof obligations each introduces.
- **An executable playbook (§7).** We recast the artifact as a guideline an autonomous language-model agent can follow to test arbitrary GUI software under this discipline, so the method generalizes beyond the one system we tested.
- **A verification agenda (§8).** We argue that this decomposition is a viable route to formal verification of interactive, networked, cloud-backed software: use exactly as much determinism and as much language model as each step requires, and make the division auditable by a certificate whose meaning depends only on declarations and datasheets.

We are explicit about status: the guarantees are conditional on the oracle model (judges being calibratable instruments), the datasheet is measured on one system’s interface families, and the extension of §6 is a design, not yet a mechanized metatheory. What is solid is the empirical core—the harness, the ten bugs, the measured operating point—and the demonstration that a real interactive test *is* a *Kimiya* program.

2 Kimiya in Brief

We recall only what this paper uses; [Anonymous \[2026\]](#) give the full calculus, denotational semantics, logic, and metatheory.

Kimiya treats a language model as a processor whose basic operations act on meaning. Its instruction core is three *oracles*, each carrying a *datasheet*—an empirically measured operating point:

- $x := \text{gen}\langle S \rangle(p)$ produces an S -schema-valid value from prompt p ; it is *untrusted* (validity v_S is syntactic; no semantic property of the output is assumed).
- $\text{judge}\langle k, \tau \rangle \psi$ evaluates a semantic predicate ψ by k oracle samples at acceptance threshold τ , with a measured confusion matrix: false-accept rate $\alpha = \Pr[\text{tt} \mid \psi \text{ false}]$, true-accept $\beta = \Pr[\text{tt} \mid \psi \text{ true}]$, and intra-panel correlation ρ .
- $x := \text{select}\langle \rho_r \rangle(q, w)$ retrieves from store w with datasheet-stamped recall $\rho_r = \Pr[i \in x \mid i \text{ relevant}]$.
- $\text{check } \varphi$ is the *trusted kernel*: a decidable predicate (schema validation, a test, a proof check) at certainty 1, cost 0.

Control flow is internalized in judged conditionals (**if** $\text{judge } \psi$ **then** C_1 **else** C_2) and *verified retry* (**retry** C **until** J **budget** B : rerun C from the loop’s entry state until judgment J passes or the budget is spent, whereupon the program *abstains* to an explicit outcome $\boxtimes$). Value identity is a *purpose-indexed judged equivalence* $a \sim_\kappa b$: reflexive, symmetric, and deliberately *non-transitive*—a *tolerance relation* of which classical equality is the zero-tolerance special case. A purpose $\kappa = \langle \text{domain}, \text{preserve}, \text{allow_loss} \rangle$ declares what an identification must preserve and what it may lose; “the same” is always “the same for a purpose.”

Programs denote maps in a cost-graded subdistribution monad—divergence is honest missing mass, budget exhaustion an explicit observable $\boxtimes$, so *failure is never silent*. On top sits a distributional Hoare logic with triples $\{P\} C \{Q\}_{\geq \theta @ B}$: from precondition P , program C commits in a Q -state with probability at least θ within budget B . The rules are parameterized by datasheet quantities; the two we use directly are:

$$\frac{(\alpha, \beta) \text{ from } d \quad \{P \wedge \psi\} C_1 \{Q\}_{\geq \theta_1} \quad \{P \wedge \neg \psi\} C_2 \{Q\}_{\geq \theta_2}}{\{P\} \text{ if judge}\langle k, \tau \rangle \psi \text{ then } C_1 \text{ else } C_2 \{Q\}_{\geq \min(\beta \theta_1, (1-\alpha) \theta_2)}} \text{ (JUDGE-IF)}$$

$$\frac{(\rho_r, \pi) \text{ from } d \quad \forall i. \{P \wedge i \in x\} C \{Q(i)\}_{\geq \theta}}{\forall i \in \text{Rel}(q, w). \{P\} x := \text{select}\langle \rho_r \rangle(q, w); C \{Q(i)\}_{\geq \rho_r \theta}} \text{ (SELECT)}$$

(For readability we omit Kimiya’s cost terms from both conclusions, and write (α, β) for what the full rules consume as conservative *effective* ends $(\alpha_{\text{eff}}, \beta_{\text{eff}})$ of a deployed panel configuration; our judges are single-sample, where the two coincide.) Two structural facts matter below. First, a judged guard transfers reliability across an *imperfect observation of a latent truth*—the branch genuinely executes under ψ or $\neg \psi$, but all a run obtains is the judge’s record. Second, a claim proved through check carries certainty 1; a claim proved through judge carries only β (or $1 - \alpha$). Kimiya keeps these two grades distinct, and that distinction is the hinge of our analysis.

3 The Harness

We summarize the artifact; the correspondence to Kimiya is §4.

Split of concerns. Scenarios are prewritten step lists (a YAML DSL). A *runner* interprets them; a *locator* wraps a mid-tier vision model (Claude Opus 4.8, invoked headlessly) behind three primitives—locate (natural-language description $\rightarrow$ bounding box and click point), judge (“does the screen show X ?” $\rightarrow$ boolean with a one-sentence reason), and read (“what value is displayed here?” $\rightarrow$ a

Table 1. The ten production bugs found by the harness. “Oracle” = caught by a non-visual check; the rest surfaced through scripted interaction plus a judged or read step.

#	Defect (one line)	Caught by
1	config key never read $\Rightarrow$ all slide data in volatile /tmp	DB oracle
2	slide-marker wiring only in the cloud branch of voice recording	audio + log oracle
3	hardcoded white input under a dark-theme white text token	screen judge
4	slide gate armed too late; exempted the app’s own frames	cloud oracle + report
5	session slide counter incremented, never decremented on delete	cloud oracle
6	session row never persisted; Sessions view showed “no talks”	scripted flow
7	viewer-count had zero callers (viewers poll; no socket to count)	cloud oracle
8	four backend modules used a never-connected DB pool	HTTP probe
9	db.transaction() called but unimplemented; non-atomic writes	exposed by #8
10	welcome bonus granted twice once #9 was fixed	exposed by #9

string). Locate results are cached by (screen size, description); a replay mode reruns whole scenarios from the cache with zero model calls. The vision model appears in exactly these three primitives and nowhere else; it never chooses the next step.

The dimensions. Beyond click and type, the harness supplies what interactive testing usually excludes. A *presenter* renders PDF pages fullscreen on a chosen monitor under a schedule, standing in for the human who presents. A *virtual microphone* (a PipeWire loopback whose playback side is a device-class source) becomes the system default input; text-to-speech (piper) speaks one line per slide, and on the output side the harness records the default sink’s monitor and asserts waveform peaks—including an inverted mode that proves *silence*. In the WebRTC voice test the speech feeds only the loopback microphone, so any energy at the speaker sink can only have crossed the mesh from the sending browser. A *relay* routes the application’s cloud traffic through a local HTTP proxy; killing it is a clean network outage. Two *browser actors* bind names to real, user-prepared Edge and Firefox windows by title; steps carrying a target crop the screenshot to that window and offset clicks into it, so two viewers of the same page cannot be confused.

Oracles. Every scenario ends in checks that are not visual: structural queries against the application’s SQLite; perceptual hashing of each stored slide against the source PDF pages (best-match, with an order constraint, because deduplication legitimately merges near-identical pages); the *public* cloud endpoint—what the audience’s viewer actually renders—for slide and viewer counts; audio-waveform peaks; and regex evidence in the application’s own logs. Artifacts of every run—a film-strip HTML report with the located box and click point on each screenshot, the application log, a consistent copy of its database, and recorded audio—are retained.

Yield. Across eighteen scenarios the harness found ten production bugs, all fixed and shipped, summarized in Table 1. Every one is an *integration* bug beyond the reach of the project’s 65-test unit suite; three were invisible on screen and caught purely by oracles; and bugs 8–10 form a cascade in which each fix exposed the next.

4 GUI Testing as a Kimiya Program

We now make the correspondence exact. The claim is not analogy but identity of structure: the harness is an informal, uncalibrated Kimiya interpreter, and its scenarios are Kimiya programs whose datasheets were left implicit.

Table 2. Harness constructs are Kimiya constructs. The right column names the datasheet quantity each carries.

Harness	Kimiya	Carries
locate(desc, screen)	$\text{select}\langle\rho_r\rangle(\text{desc}, \text{screen})$	recall ρ_r
assert_screen(desc)	$\text{judge}\langle k, \tau \rangle \phi_k(\text{screen})$	(α, β, ρ)
read_text(desc)→ \$v	$\text{gen}\langle \text{Str} \rangle(\text{screen}), \text{grounded downstream}$	validity v
verify_db / verify_cloud	check φ (trusted kernel)	certainty 1
verify_slides_match_pdf	check over a tolerance relation $\sim_{\kappa_{\text{img}}}$	certainty 1
verify_audio_out	check φ (peak/silence predicate)	certainty 1
click retries, optional:, re-run-once	retry · until · budget ·	—
scenario FAIL + artifacts	abstain / ✖ (explicit, recorded)	—
scenario PASS	commit of a triple $\{P\} \cdot \{Q\}_{\geq \theta}$	—
model calls per scenario	cost in the budget monoid	—

4.1 The mapping

Table 2 is the core observation. Three entries deserve comment.

Assertions are judgments under a purpose. A harness assertion reads “the live presentation view with a LIVE badge and an End Presentation button.” This is a rubric predicate $\phi_k(\text{screen})$ whose purpose κ is implicit in the wording: it *preserves* the named widgets and key text, and *allows loss* of theme, cursor, clock, and unrelated windows. The harness’s very robustness—the same assertion passing in light and dark theme, at different window sizes, with the operator’s terminal visible beside the target—is exactly what a Kimiya purpose licenses, and what a pixel-diff oracle forbids. Making κ explicit (§4.3) is the discipline the harness was missing.

Perceptual slide matching is a tolerance relation. The oracle `verify_slides_match_pdf` accepts a stored slide s as a capture of source page p when their perceptual hashes lie within a distance threshold, $s \sim_{\kappa_{\text{img}}} p$. This relation is reflexive, symmetric, and *non-transitive*: two frames each within distance 16 of a shared middle can be 24 apart. The oracle’s design— best-match against all pages plus a presentation-order constraint, rather than positional $k \leftrightarrow k$ matching—is precisely a refusal to chain the tolerance relation, the operational form of Kimiya’s non-transitivity axiom. We did not know we were obeying the axiom; the axiom is what makes the oracle correct.

Reads are untrusted generation, trusted downstream. When the harness reads an 8-character group join code off browser A’s success dialog into a variable and types it into browser B, that read is a gen: untrusted. We never relied on it. Its correctness was established *downstream*, by B actually joining and a check against the database membership count—exactly Kimiya’s “untrusted generator proposes, trusted kernel disposes.”

4.2 Why the harness almost never issued a false certificate

The most consequential structural fact is why, across roughly thirty runs, the vision layer produced *zero* false passes. In Kimiya terms, scenarios *end in check*. A judged guard (`assert_screen`, `locate`) gates *progress*, not the verdict. A judge false-accept (α) merely lets the script proceed; the committed claim—cloud == kept, per-talk sequence 1..N, a WAV peak above threshold—is check-graded at certainty 1. So judge error converts into wasted budget and abstention, almost never into a false certificate. This is precisely what JUDGE-IF composed with CHECK predicts: the reliability that reaches the commit is the product of a judge factor and a kernel factor, and when the kernel factor is 1 on the load-bearing postcondition, the certificate is robust to α . It also explains the observed failure profile: failures were overwhelmingly *correct rejections*—a real bug, real state drift from

a prior failed run, or a wrong assertion—rather than silent mispasses. The harness was, without knowing it, exploiting the check/judge grade separation that is Kimiya’s central safety property.

A corollary sharpens honesty. Not every harness postcondition is kernel-graded. “The ink stroke renders in browser B” and “the LIVE badge is shown” are judge-graded: certified only at β . A Kimiya reading of a passing run therefore *stratifies* its claims—which hold at certainty 1 (counts, peaks, cloud totals) and which hold only at the judge’s operating point—where the current PASS conflates them. That stratification is the paper’s practical upgrade to the harness, and it is where an undetected α -error could still hide a product bug.

4.3 A scenario, transcribed

We transcribe the two-user collaboration scenario (create a group in A, carry the join code to B, verify chat both ways) into Kimiya surface syntax, purposes made explicit. `act` and `settle` are the world-effecting constructs proposed in §6 (read `act` as “perform this input effect”); everything else is core Kimiya, with the purpose of each `select` noted in a comment exactly as the Kimiya overview does.

```
context k_ui:                                -- locating a described control
  domain      = "GUI control matching a description on a screenshot"
  preserve    = [role, visible_label, affordance]
  allow_loss  = [pixel_position, theme, scale, unrelated_windows]

context k_state:                             -- judging an application state
  domain      = "whether a screenshot shows a described UI state"
  preserve    = [key_widgets, key_text, key_status]
  allow_loss  = [cursor, clock, theme, unrelated_windows]

-- A creates the group; the join code is generated (untrusted) and read.
act<A> click( select<0.97>("the + button by the groups heading", A.screen) ) -- under k_ui
act<A> click( select<0.97>("the Create Group button", A.screen) )           -- under k_ui
act<A> type ( "W3 Harness Group" )
act<A> click( select<0.97>("the Create submit button", A.screen) )         -- under k_ui
settle<A> until judge<1,1/1> shows(A.screen,
    "a success view with an 8-char join code") under k_state
    within 12s

code := gen<JoinCode>( read("the 8-character join code", A.screen) ) -- untrusted

-- B joins with the carried code.
act<B> click( select<0.97>("the + button by the groups heading", B.screen) ) -- under k_ui
act<B> click( select<0.97>("the Join Group button", B.screen) )             -- under k_ui
act<B> click( select<0.97>("the Join Code input", B.screen) )              -- under k_ui
act<B> type ( code ); act<B> key ( Return )
settle<B> until judge<1,1/1> shows(B.screen,
    "the group 'W3 Harness Group' is selected") under k_state
    within 12s

-- chat A -> B, then B -> A, each verified on the receiving screen ...
-- (omitted; identical shape)
```

```
-- THE VERDICT: kernel gates, then a judged gate, then commit.
if check( db.group_members("W3 Harness Group") = 2           -- kernel: certainty 1
        /\ db.messages(group) = [m_ab, m_ba] )               -- kernel: certainty 1
then if judge<1,1/1> shows(B.screen, "the chat pane displays m_ab")
    under k_state                                           -- judged: >= beta
    then commit(trace) else abstain
else abstain
```

Read the final block: it makes explicit what the informal PASS left tacit. A run commits only through two gates. The kernel gate (check on the membership count and message rows) contributes no instrument error: *any* committing run satisfies those conjuncts with certainty. The judged gate (“the chat pane displays m_{ab} ”) is an instrument reading at the locator’s operating point; the certificate carries that conjunct at the datasheet rate, not at 1. A reviewer of the test—or an LLM agent asked whether the collaboration feature is verified—learns from the certificate exactly which guarantees are ironclad and which are probabilistic, and at what rate.

4.4 The reliability of a passing run, numerically

We instantiate the Kimiya logic on a passing scenario using the *measured* datasheet of §5. Consider a scenario with L load-bearing locate steps (each gating progress toward the commit), J judged guards on the path to the commit, and a commit whose postcondition is a conjunction of kernel checks ϕ_{ker} and judged claims ψ_{jud} . Under the harness retry policy (each locate retried, plus a scenario re-run on terminal miss), the per-call locate recall ρ_r composes to an effective $\rho_r^{\text{eff}} \approx 1$ over the campaign (§5); we carry ρ_r symbolically.

The kernel conjunct is certified at 1 by CHECK. Each judged guard on the path contributes its true-accept factor β (the guard must fire for the run to reach commit; JUDGE-IF on the taken branch). Each judged *postcondition* claim contributes β directly. So the certified reliability of the commit factors as

$$\theta_{\text{commit}} \geq \underbrace{(\rho_r^{\text{eff}})^L}_{\text{located}} \cdot \underbrace{\beta^J}_{\text{guards fired}} \cdot \underbrace{1}_{\phi_{\text{ker}}} \cdot \underbrace{\beta^{|\psi_{\text{jud}}|}}_{\text{judged claims}}.$$

For the transcribed scenario, the load-bearing kernel conjuncts (member count, message rows) are the guarantee that matters, and they are certified at 1 regardless of β —this is the formal content of §4.2. The *judged* conjunct (“B’s chat shows m_{ab} ”) is certified at $\beta \geq 0.976$ (§5). If one instead demanded a single reliability figure for the *whole* commit read conjunctively, with $J = 4$ guards on the path and one judged postcondition, and taking the conservative $\beta = 0.976$ and $\rho_r^{\text{eff}} = 0.999$ over $L = 8$ locates, the composed lower bound is

$$\theta_{\text{commit}} \geq 0.999^8 \cdot 0.976^4 \cdot 1 \cdot 0.976 = 0.992 \cdot 0.908 \cdot 0.976 \approx 0.879,$$

valid with datasheet-confidence $1 - \delta$ (§5 publishes the locator sheet at the campaign’s aggregate level). The number is deliberately unglamorous, and that is the point: a green run is not certainty, it is a priced guarantee, and the price is dominated by how many *judged* claims ride on the commit versus how many are discharged to the kernel. The design lesson the arithmetic hands back to the test author is exactly the one §4.2 states: push load-bearing postconditions into check, and the $\beta^{(\cdot)}$ factors stop mattering.

5 A Measured Datasheet for the Vision Locator

A Kimiya instrument is only as trustworthy as its datasheet, and the datasheet must be *measured*, not assumed. We calibrate the harness’s vision locator (Claude Opus 4.8 via a headless CLI) from the

Table 3. Measured operating point of the vision locator, from 58 runs / 604 model calls. Bounds are conservative (Wilson-95 lower for accept rates; rule-of-three upper for unobserved error).

Instrument	Quantity	Value	Basis
judge (assert)	β true-accept	≥ 0.976	158/158; Wilson-95 low
judge (assert)	α false-accept	≤ 0.16	0/19 true-neg; rule of three
select (locate)	ρ_r per call	≥ 0.975	340/343; Wilson-95 low
select (locate)	ρ_r^{eff} after retry	≈ 1.000	0 unrecovered / 58 runs
gen (read)	value error	≤ 1.0	0/3; rule of three (thin)
cost	latency p_{50}/p_{90}	8.9/14.3 s	229 samples
cost	calls / scenario	10.4	604/58

campaign’s own retained artifacts: 58 scenario runs with structured step reports, and the consoles of 55 runs recording per-call latency and attempt-level retry lines. Ground truth is established in two tiers, reported separately: *(T1) outcome-grounding*—a step’s latent truth inferred from the run’s terminal hard oracles, which are independent of the vision layer; and *(T2) per-event adjudication*—every non-passing perception event in the campaign was root-caused when it occurred (product bug, oracle bug, state drift, or model flake), and that record is reused here. Where zero errors were observed we report the one-sided 95% upper bound (rule of three, $3/n$), never zero.

Table 3 is the result. Three points are worth stating plainly, in the datasheet spirit of admitting what is weak.

The false-accept bound is the honest limitation. We observed no judge false-accept—including a blind re-read of six uniformly sampled accepted assertions, all confirmed correct—but the campaign presented only ≈ 19 genuine true-negative trials to the assertion judge (runs where a described state was truly absent and correctly rejected). The rule of three therefore caps $\alpha \leq 0.16$, no tighter. This is exactly the quantity §4.2 shows the harness is *robust* to (because commits end in check), which is fortunate, because it is the quantity we can least tightly bound from opportunistic data. A purpose-built calibration suite—the standard Kimiya remedy—would drive it down.

Correlation is real and unmeasured. The configuration is single-model, single-prompt; immediate same-input retries are highly correlated (we observed three consecutive misses on one plainly visible element, then success on a later fresh run). We therefore treat an immediate retry burst as *one* effective sample and rely on the scenario re-run—a genuinely fresh draw—for amplification. This is the uncalibrated shadow of Kimiya’s panel construct (panel [A,B,C] [paraphrase_prompts](#) 2): decorrelation by model family and prompt paraphrase is what would let $k > 1$ actually amplify, and it is the first thing a production version should adopt.

Recall is the certificate-bearing direction. Per-call locate recall is high (≥ 0.975) and, under the retry policy, effectively 1 over the campaign (no unrecovered locate failure caused a false result). This matches Kimiya’s asymmetry: a locate *miss* is loud (the step retries or the scenario abstains, visibly), whereas a locate on the wrong element is caught downstream when the subsequent judged/kernel step fails. Recall, not precision, is what the certificate rides on, exactly as the select datasheet demands.

6 Extending Kimiya for Effectful, Timed Interaction

The correspondence of §4 is faithful for the verification layer but strains at the interaction layer, because GUI testing is Kimiya’s deferred extension in its most demanding form: computation that *mutates an external world under adversarial timing*. We identify three gaps and propose the

constructs and proof obligations a world-effecting Kimiya needs. We present this as design, with the metatheory sketched, not mechanized.

6.1 Gap 1: the store is an external, mutating world

Kimiya’s state is $\Sigma = \text{Var} \rightarrow \mathbb{V}$: values under the program’s control. The harness’s state is a live desktop, two browsers, an audio server, and a production database—an external store $\mathcal{W}$ the program can *act on* and *observe* but not directly read. We propose splitting the interaction primitives:

- act e : perform an input effect on $\mathcal{W}$ (a click at a located point, a keystroke, an audio injection). Its datasheet is a *delivery* contract—probability the effect is delivered, and its cost—not a truth about the resulting state.
- observe(S): sample a schema-typed observation of $\mathcal{W}$ (a screenshot, a DB row, a waveform). An observe is the *subject* of a subsequent judge or check; it is where latent world-truth enters as an instrument reading.

The locate-then-click idiom becomes `act click(select( $\rho_r$ )(desc, observe(screen)))`: observe the screen, retrieve a point on it, act. Crucially, select now retrieves over an *observation*, so its recall datasheet is conditioned on the observation family—which is exactly how the harness’s locator behaves (its reliability differs across the PyQt, GTK-dialog, and browser surface families of Table 3).

6.2 Gap 2: retry must reason about world frames

Kimiya’s `RETRY` reruns the body *from the loop’s entry state*—snapshot semantics—which is sound because the store is program-owned and can be reset. An external world cannot be snapshotted: a click that landed wrong may have opened a dialog or joined a call, and the next attempt inherits it. This is not a corner case; it is the source of the harness’s hardest-won engineering (optional steps, leave-then-rejoin, the “dead microphone node” hazard). A world-effecting retry needs a *frame condition* on the body:

$$\frac{\begin{array}{ll} \{P\} C \{Q\} \text{ on success} & C \text{ is } \mathcal{W}\text{-idempotent up to } \phi_{\text{inv}} \\ (\alpha_{\text{eff}}, \beta_{\text{eff}}) \text{ for } J & \text{on failure } C \text{ runs compensate restoring } \phi_{\text{inv}} \end{array}}{\{P \wedge \phi_{\text{inv}}\} \text{ retry } C \text{ until } J \text{ budget } B \{Q\}_{\geq (1 - (1 - g\beta_{\text{eff}})^B) \pi_{\text{acc}}}} \quad (\text{WORLD-RETRY})$$

Here g is the body’s datasheet task-family quality (per-round probability of producing a Q -satisfying outcome) and $\pi_{\text{acc}} = g\beta_{\text{eff}} / (g\beta_{\text{eff}} + (1 - g)\alpha_{\text{eff}})$ the acceptance precision, both exactly as in Kimiya’s `RETRY` rule [Anonymous 2026]; only the premises are new. They are the price of an unresettable store. Either the body is *idempotent up to a declared world-invariant* ϕ_{inv} (re-running a failed click leaves $\mathcal{W}$ within ϕ_{inv} of where it began—the property the harness’s `optional`: steps in informally assert), or it carries an explicit compensate action that re-establishes ϕ_{inv} before the next round (the formal version of leave-then-rejoin). ϕ_{inv} itself is a check- or judge-checkable predicate on an observe; when it is judged, its error enters the reliability like any other. The obligation that this premise cannot be met silently—that a body which mutates the world outside ϕ_{inv} is a *type error*, not a runtime surprise—is the interactive analogue of Kimiya’s ban on silent semantic equality.

6.3 Gap 3: cost is timed, and time is adversarial

Kimiya’s cost monoid prices *calls*; the harness’s sharpest methodology rule—never walk playback markers while audio plays, because the locator’s seconds of latency are wall-clock drift—has no home in it. The world evolves *while the judge thinks*. We propose a settle `until J within  $\delta$` construct: repeatedly observe-and-judge until J holds or a real-time deadline δ elapses, with the deadline a first-class cost alongside the call budget. settle is the principled form of the harness’s `wait+`

assert_screen idiom, and its obligation is a *stability* premise: the judged state must be one the world will hold still for at least the judge’s own latency, or the reading is of a state already gone. The impossibility this formalizes—that no amount of judging can certify a claim about a state that does not persist as long as the judgment takes—is the timing analogue of the recall asymmetry: some failures are silent unless the construct forbids them.

6.4 What survives unchanged

The verification layer is untouched by all three gaps. Purposes, the check/judge grade split, budgets, abstention to $\boxtimes$, and audit locality apply verbatim; the transcribed certificate of §4.3 is already written in the extended surface syntax and its `commit` means exactly what the core logic says. The extension is entirely in *how state is touched*, not in *how reliability is proved*—which is why the harness could obey the verification discipline (and reap its safety) while improvising the interaction discipline (and paying for it in engineering).

7 A Playbook for an Autonomous Testing Agent

The method is not tied to SeenSlide. We state it as a procedure an autonomous language-model agent can follow to test arbitrary GUI software under the same discipline. It is deliberately imperative and checkable; each step names the Kimiya construct it produces.

- (1) **Fix the control flow first; never improvise it.** Write the scenario as a deterministic step list (a user story: open X , do Y , then Z). The agent may *author* the script using its judgment, but the *run* must replay it exactly. A step list is a reviewable artifact; an exploration trace is not.
- (2) **Make every meaning-laden step a named instrument.** Locating a control is `select` (a description + a recall expectation); judging a state is `judge` (a description + a purpose); reading a value is `gen` (untrusted until grounded). Do not let the model choose the next *action*—only perceive.
- (3) **Declare a purpose for every judgment.** State, per assertion, what must be preserved (the widgets/text that make the state that state) and what may be lost (theme, position, clock, unrelated windows). This is what makes the assertion robust and well-posed; an assertion without a purpose is a pixel-diff in disguise.
- (4) **End every scenario in a non-visual kernel check.** A test that concludes on a screen judgment certifies nothing at certainty 1. Find the ground truth the feature is *about*—a database row, a public API response, a file on disk, an audio waveform, a log line—and assert it with `check`. This is the single most important rule: it is what makes judge error convert to abstention rather than a false pass (§4.2).
- (5) **Stratify the certificate.** In the `commit`, separate kernel-graded conjuncts (certainty 1) from judged conjuncts ($\geq \beta$). Report both. A consumer of the test—human or agent—must know which guarantees are ironclad.
- (6) **Retry with a declared world-invariant.** Wrap flaky interaction in `retry/optional`, but state what re-running is allowed to disturb (§6): either the action is idempotent up to ϕ_{inv} , or it carries a compensation. Treat a call surviving from a prior run (a dialog, a joined call, a dead device handle) as inherited world state, not a clean slate.
- (7) **Wait with settle, not a fixed sleep, and never judge a transient.** Poll-observe-judge until the state holds or a real-time deadline passes. Do not perceive during continuous change (playback, animation, streaming); the perception latency will read a state already gone.

- (8) **Isolate and neutralize side effects.** Run the app under a disposable environment (a fresh home directory, a throwaway identity, a null credential store). For cloud tests, ensure the test identity cannot touch real user data, and purge test artifacts with a filter that names their test origin *in the deletion itself*—never a list gathered from logs. Route network through an injectable proxy so outages are a test action, not a host change.
- (9) **Measure your instruments before you trust them.** Retain every run’s screenshots, judgments, and outcomes. Periodically re-adjudicate a sample against the terminal kernel oracles to estimate (α, β, ρ_r) and latency; publish the datasheet; recompute the scenario’s certified reliability (§4.4). Report unobserved error as a one-sided bound, never as zero.
- (10) **Decorrelate to amplify.** A single model retried on the same prompt barely amplifies. When a judgment is load-bearing and cannot be pushed to the kernel, vote across model families and prompt paraphrases (a panel), and treat an immediate same-input retry burst as one effective sample.

Steps 4 and 5 are the discipline’s core; a large-language-model agent that follows only those two already tests far more safely than an autonomous explorer, because its verdict stops depending on its own eyes. The remaining steps buy tighter numbers and safer interaction.

8 Toward Verified Interactive Systems

The wider motivation is a route to *formal verification of interactive, networked, cloud-backed software*—systems that have long resisted it for two reasons this decomposition addresses head-on.

Such systems resist verification because parts of their specification are *semantic*—“the viewer shows the presenter’s current slide,” “the audience hears the speaker,” “the schedule was imported correctly”—and no decidable predicate captures them; and because their state is an *external world* no proof assistant can model in full. The usual responses each fail one half: classical model checking demands the whole system reduce to a finite transition system, which the semantics and the world both refuse; and pure LLM-agent testing certifies nothing, because an improvising agent’s judgment is uncalibrated and its runs are not artifacts.

A Kimiya program is a third option, and the harness is its existence proof. It uses *exactly as much determinism as the task requires*—the control flow, the kernel oracles, the world-invariants—and *exactly as much language model as the task requires*—the locates, the state judgments, the grounded reads—and the boundary between them is not a matter of taste but of what each step *can* be: decidable predicates go to check, meaning-laden ones to the instruments. The certificate then states precisely what was verified at certainty 1, what was verified at a measured error rate, and what was left uncovered—and, by audit locality, its meaning depends only on the program’s declarations and the cited datasheets, so a reviewer (or a downstream agent) can trust a test without replaying it.

Concretely, this suggests a research program. (i) A world-effecting Kimiya core (§6) with a mechanized soundness theorem for WORLD-RETRY and settle, reducing the external world to an oracle-observed store with declared invariants. (ii) A calibration methodology for interface-perception instruments —purpose-built suites per surface family (native toolkit, web, mobile), published as datasheets an agent can cite. (iii) A compiler from user-story specifications to Kimiya test programs, closing the loop the playbook opens by hand. The end state is a testing regime in which a green run is not a boolean but a signed certificate—“this feature holds at certainty 1 on these kernel facts and at $\geq \beta$ on these judged facts, within this budget, citing these datasheets”—for exactly the software that today has no verification story at all.

9 Related Work

LLM-driven GUI testing is predominantly mobile and autonomous: scenario-guided and multi-agent exploration *generate* actions, where we execute fixed scripts and delegate only perception; desktop is a recognized gap. *Commercial AI test tools* offer natural-language authoring and self-healing selectors, philosophically akin to our select, but scoped to web/mobile and silent on audio, native dialogs, and cross-application flows. *Multi-user feature testing* coordinates LLM agents on separate mobile devices; we test two-user collaboration deterministically, with the model reading a runtime-generated code across browsers. *Voice/WebRTC testing* evaluates STT/LLM/TTS pipelines, typically via fake media streams; we found no system closing the audio loop at the operating-system level inside a general GUI harness. Foundationally, we build on Kimiya [Anonymous 2026]: our contribution is to show that an independently constructed interactive test is a Kimiya program, to measure a real instrument datasheet, and to identify the world-effecting extension the domain forces. (A full bibliography accompanies the camera-ready; the companion technical report enumerates the harness’s ten bugs and scenarios in full.)

10 Limitations

The guarantees are conditional on the oracle model—judges being calibratable instruments—which our datasheet supports in a restricted regime (short, grounded, discriminative visual judgments) and no further. The datasheet is measured on one product’s interface families and one model; the α bound is loose for want of true-negative trials (§5), and correlation is treated conservatively rather than measured. The world-effecting extension (§6) is a design with sketched obligations, not a mechanized metatheory. The harness is Linux/X11-specific in its input and capture layers, requires exclusive use of the screen, and cannot test sub-second interactive behavior because perception latency is seconds. Finally, our evidence is one system and one operator: the ten bugs are an existence proof of effectiveness, not a controlled comparison. What we claim is narrow and, we believe, solid: that a real end-to-end interactive test, built without knowledge of the theory, turned out to be a Kimiya program, and that reading it as one both explains its safety and shows how to make it a certificate.

11 Conclusion

We set out to test a real interactive product and, in doing so, built a harness that drives desktop and dual-browser surfaces, closed-loop audio, network faults, and a live cloud from one deterministic step vocabulary, finding ten shipped bugs. Reading the result through Kimiya, we found the harness had implemented the theory unaware: its locator is select, its assertions are judge under implicit purposes, its reads are grounded gen, and its database and audio oracles are the check kernel whose presence is precisely why judge error almost never became a false pass. We measured the locator’s operating point from the run artifacts, derived a numeric certificate for a passing test, and identified the one place the correspondence strains—the external, timed world—proposing the constructs a world-effecting Kimiya needs. The method generalizes to a playbook any language-model agent can follow. We read all of this as a first, concrete step toward verifying the interactive, networked, cloud-backed systems that have no verification story today: use exactly the determinism and exactly the language model each step demands, and let a Kimiya certificate make the division auditable.

References

Anonymous. 2026. Kimiya: A Program Logic for Semantic Computation with Language Models. *Manuscript*.